DD2424 — Assignment 1 Report

One-Layer Softmax Classifier on CIFAR-10

1. Introduction

This report describes the implementation and evaluation of a single-layer softmax classifier trained on the CIFAR-10 dataset using mini-batch gradient descent. The model maps each 3072-dimensional image vector to a probability distribution over 10 classes via a linear transformation followed by softmax. The cost function combines the mean cross-entropy loss with an L_2 regularization term on the weight matrix W. Four hyperparameter configurations are explored to study the effect of learning rate and regularization strength.

2. Gradient Verification

The analytic gradients computed by **BackwardPass** were verified against gradients obtained from PyTorch's automatic differentiation (**ComputeGradsWithTorch**). The check was performed on a reduced problem (d=10, n=3) for both λ=0 and λ=0.1. Results are summarized below.

	Max Abs Error (W)	Max Abs Error (b)	Max Rel Error (W)	Max Rel Error (b)
λ = 0	1.11e–16	5.55e–17	2.10e–15	1.19e–16
λ = 0.1	1.11e–16	5.55e–17	4.58e–15	1.19e–16

Table 1. Gradient verification results. All errors are at machine precision level.

All absolute errors are on the order of 10^{-16} and relative errors on the order of 10^{-15}, confirming that the analytic gradient computation is correct. The check was repeated with λ=0.1 to verify the regularization gradient term ($2\lambda W$), which also passed with the same precision.

3. Training Results

The network was trained with four hyperparameter configurations, all using n_batch=100 and n_epochs=40. Training data: data_batch_1 (10,000 images), validation: data_batch_2, test: test_batch. Data was normalized per dimension using training set statistics. Weights were initialized with Gaussian noise (σ=0.01, seed=42).

Config	λ	η	Train Acc	Val Acc	Test Acc
1	0.0	0.1	42.12%	27.10%	27.70%
2	0.0	0.001	45.57%	38.46%	39.21%
3	0.1	0.001	44.63%	38.62%	39.30%
4	1.0	0.001	39.85%	36.32%	37.55%

Table 2. Final accuracies after 40 epochs for each configuration.

3.1 Configuration 1: λ=0, η=0.1

With a learning rate of 0.1, the training is highly unstable. The loss oscillates wildly between 3 and 10 across epochs, never settling into smooth convergence. The large gap between training accuracy (42%) and validation accuracy (27%) indicates severe overfitting compounded by gradient instability. The learned weight templates appear as random noise, confirming the network has not extracted meaningful class-discriminative features. The test accuracy of 27.70% is only marginally above the 10% random baseline, demonstrating that an excessively high learning rate prevents effective learning.

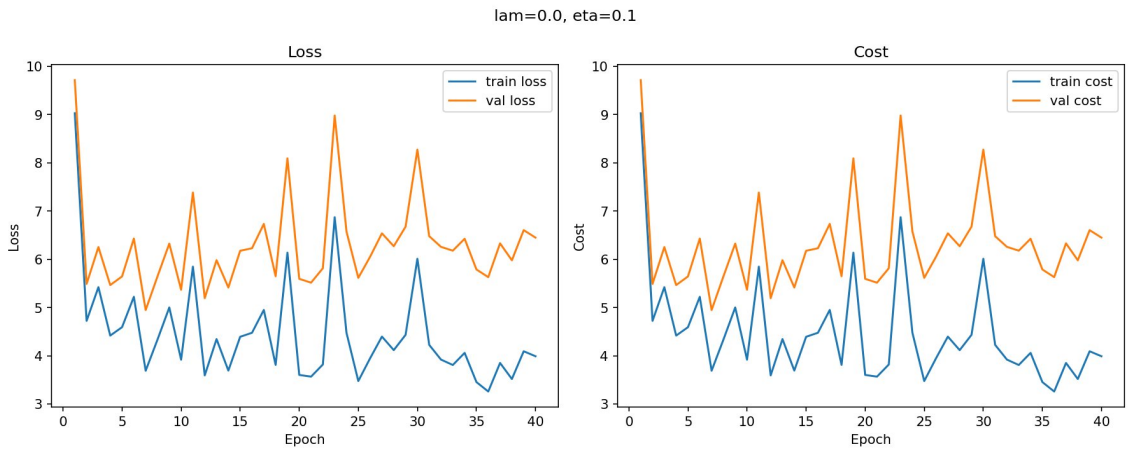

Figure 1. Loss and cost curves for λ=0, η=0.1. Highly unstable training.

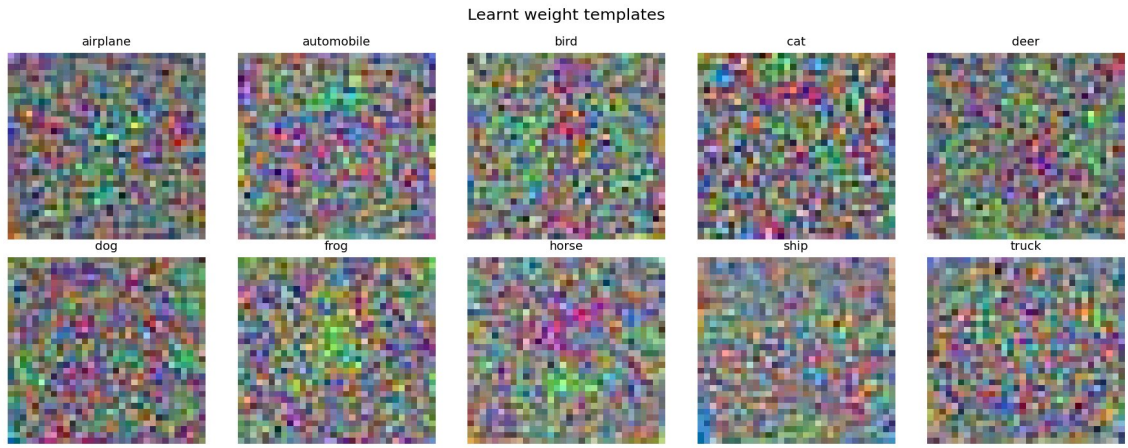

Figure 2. Weight templates for λ=0, η=0.1. No meaningful patterns learned.

3.2 Configuration 2: λ=0, η=0.001

Reducing the learning rate to 0.001 produces smooth, monotonically decreasing loss curves. The training loss reaches 1.61 while the validation loss plateaus around 1.79, revealing a growing generalization gap characteristic of zero regularization. Since λ=0 the loss and cost plots are identical. The weight templates remain fairly noisy because unconstrained weights can take on large, high-frequency values. The test accuracy of 39.21% is close to the instructor's reference of 39.13%.

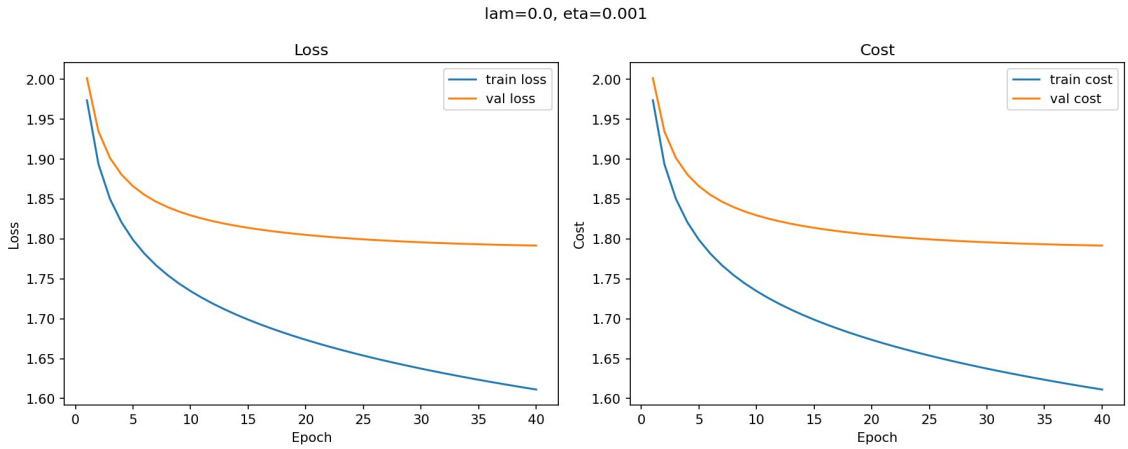

Figure 3. Loss and cost curves for λ=0, η=0.001. Smooth convergence, some overfitting.

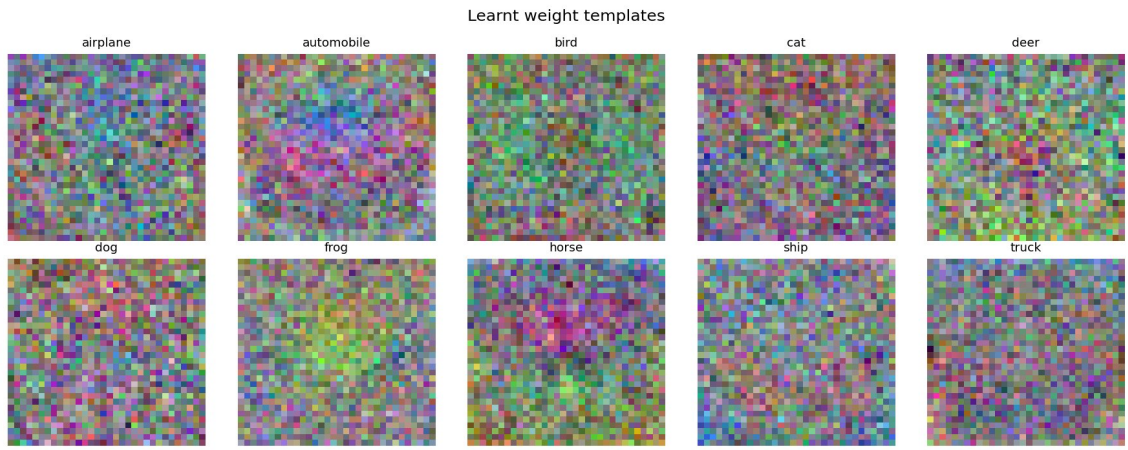

Figure 4. Weight templates for λ=0, η=0.001. Noisy templates due to no regularization.

3.3 Configuration 3: λ=0.1, η=0.001

Adding moderate regularization (λ=0.1) yields loss curves very similar to Config 2, with the key difference visible in the cost plot: the regularization penalty creates a gap between cost and loss, and the train/val cost curves are closer together than in the unregularized case. The loss curves (which exclude the penalty) show that the gap between train and validation loss is slightly reduced compared to Config 2, confirming that regularization mitigates overfitting. The weight templates begin to show slightly more coherent patterns. Test accuracy improves marginally to 39.30%.

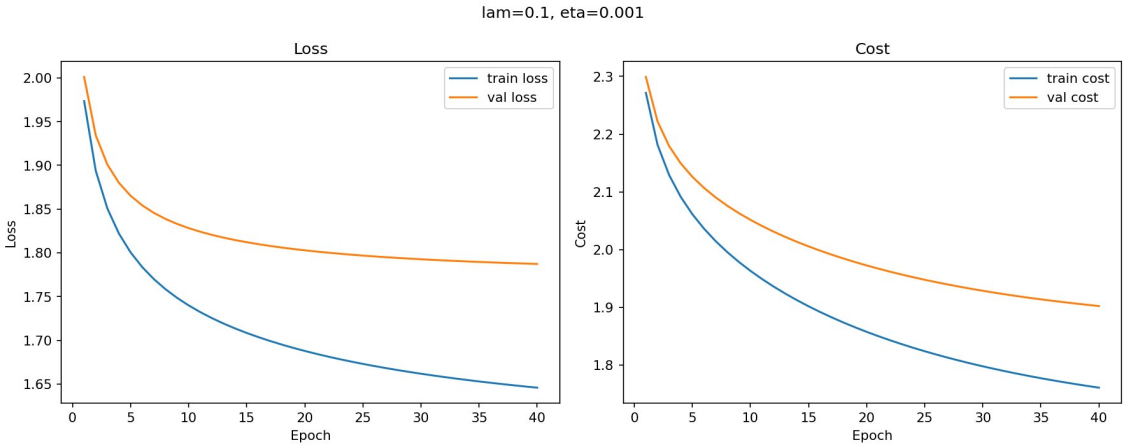

Figure 5. Loss and cost curves for λ=0.1, η=0.001. Cost includes the regularization term.

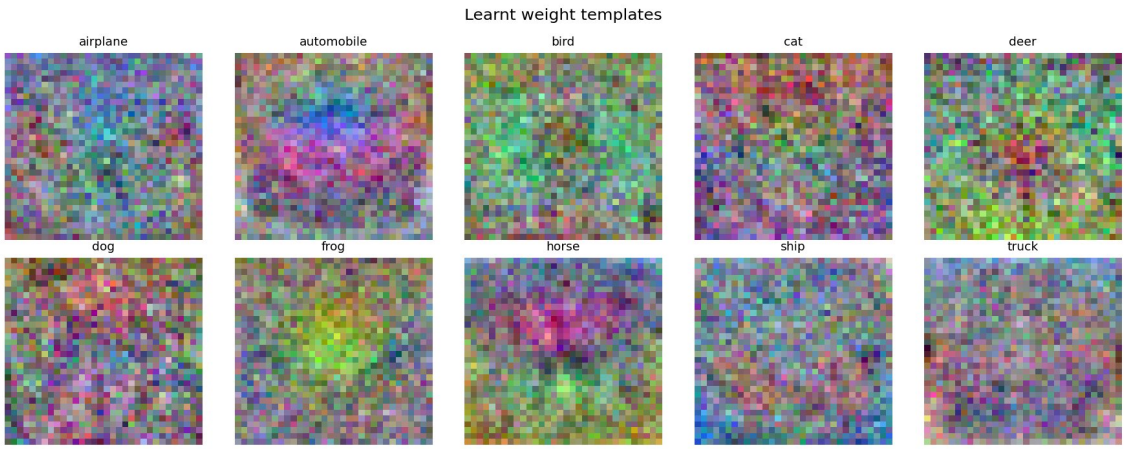

Figure 6. Weight templates for λ=0.1, η=0.001. Slightly more structured patterns emerging.

3.4 Configuration 4: λ=1.0, η=0.001

Strong regularization (λ=1.0) produces the most striking differences. The train and validation loss curves nearly overlap, showing that overfitting has been almost entirely eliminated. However, the loss plateaus early (around epoch 15) at a higher value (~1.80) compared to Configs 2 and 3 (~1.61 train loss), indicating underfitting — the heavy penalty prevents the weights from growing large enough to discriminate well between classes. The cost starts very high (~4.0) due to the large initial regularization penalty on the randomly initialized weights, then drops rapidly as the weights shrink. The weight templates are now clearly interpretable: the airplane template shows a blue sky, the frog is bright green, the ship has a blue water background, and the horse shows a brown body shape. Test accuracy drops to 37.55%, reflecting the underfitting tradeoff.

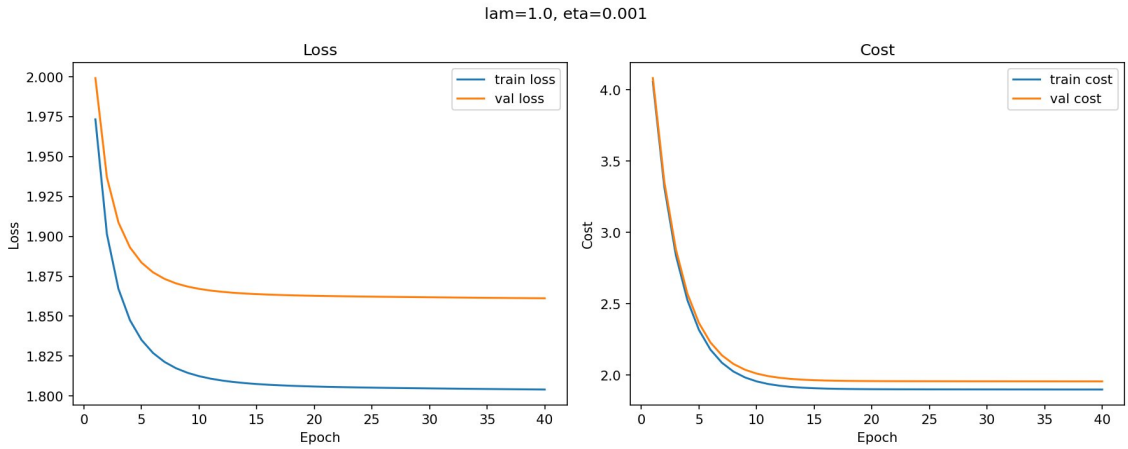

Figure 7. Loss and cost curves for λ=1.0, η=0.001. Minimal overfitting but early plateau.

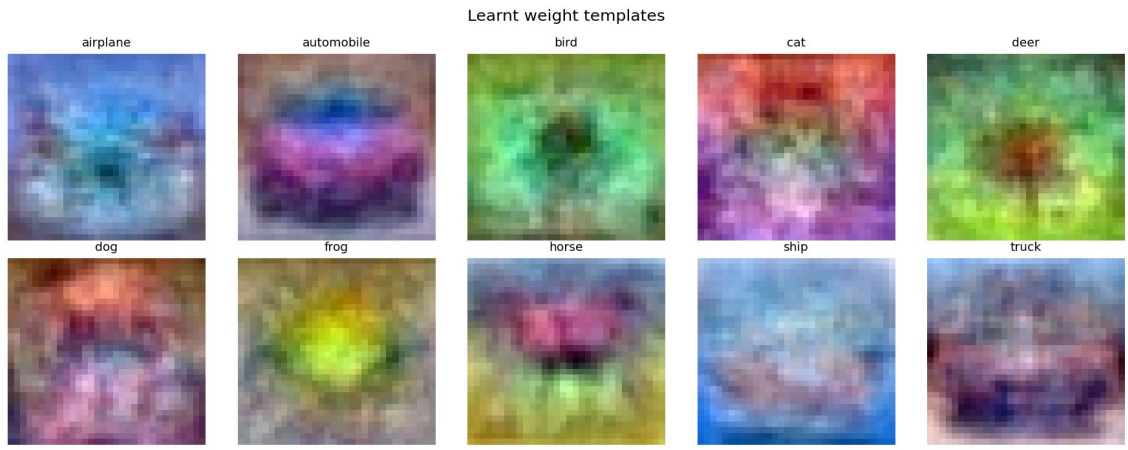

Figure 8. Weight templates for λ=1.0, η=0.001. Highly interpretable class templates.

4. Discussion

Effect of the Learning Rate

The comparison between Configs 1 and 2 demonstrates the critical importance of selecting an appropriate learning rate. A learning rate of 0.1 is far too large for this problem, causing the optimization to overshoot minima and oscillate erratically. Reducing it to 0.001 produces stable, monotonic convergence. This illustrates that while larger learning rates can in principle speed up training, they risk divergence if they exceed the curvature of the loss surface.

Effect of Regularization

Comparing Configs 2, 3 and 4 reveals the regularization tradeoff. Without regularization (λ=0), the model overfits: training loss continues to decrease while validation loss stagnates, and the weight templates are noisy. Moderate regularization (λ=0.1) slightly reduces the generalization gap and produces marginally better test accuracy (39.30% vs 39.21%). Strong regularization (λ=1.0) eliminates overfitting entirely but constrains the model so heavily that it underfits, lowering test accuracy to 37.55%. The most notable benefit of strong regularization is the interpretability of the weight templates, which become recognizable class prototypes. This reflects the fact that L_2 regularization penalizes large weights, forcing the model to rely on smooth, low-frequency features rather than fitting noise in the training data.

5. Conclusion

All required components of Assignment 1 have been implemented and verified. The analytic gradient computation matches PyTorch's automatic differentiation to machine precision. The best test accuracy achieved is 39.30% with λ=0.1 and η=0.001, which is consistent with the instructor's reference result. The experiments clearly demonstrate that (1) choosing an appropriate learning rate is essential for stable convergence, and (2) regularization controls the tradeoff between overfitting and underfitting, with moderate values providing the best generalization for this single-layer model.